Computer Programming 1

Fundamental types (precision)

(28-09-2026)

Outline

- Fundamental types (precision)

Fundamental types (precision)

- Fundamental types are:
  - Boolean
  - Integer
  - Real
  - Enum
  - Char

Integer and real types

- Integer
  - short (16 bits [-32768; 32767] (standard: at least 16bits))
  - int (32 bits [-2147483648; 2147483647] (standard: at least 16bits))
  - long (64 bits [-9223372036854775808; 9223372036854775807] (standard: at least 32bits))
  - long long (64 bits [-9223372036854775808; 9223372036854775807] (standard: at least 64bits))
- Real
  - float (32bits, precision: 7 decimal digits)
  - double (64bits, precision: 15 decimal digits)
  - long double (96bits, precision: at least 15 decimal digits)

Controlling the Precision

cout prints 6 or 7 digits (it depends on the implementation), both integer and decimal part

cout.precision();

- It returns the used precision
- streamsize psize= cout.precision();
  - to store the precision (e.g., default is 6/7) in the variable psize
  - streamsize is a "special" signed integral type used only for this purpose

E.g.

```
cout.precision(10);
cout << fixed << sqrt(2) << endl;
cout << defaultfloat << endl;
```

cout.precision(*n*)

- It specifies the precision of the output to be used
- *n* is the number of digits to print on the terminal (sum between the integer and decimal part);

fixed/scientific

- it specifies the type of notation to be used
- if fixed is used with precision(n) specifies that the precision has to be computed only with the decimal part

defaultfloat

- It restores the default (fixed/scientific) notation and way of computing the precision (both integer and decimal parts)

Alternative

setprecision(X): to show X digits in the standard output (both decimal and integer part)

- the library (#include) <iomanip> is required

Controlling the Precision: example (1)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include <iomanip>
4.  int main ()
5.  {
6.   float fl1 = 3.14159265358979f, fl2 = 133.14159265358979f;
7.   double db1 = 3.14159265358979, db2 = 133.14159265358979;
8.
9.   streamsize ssize = cout.precision();  // store the default precision
10.  cout << "(1)  precision default " << ssize << endl;
11.  cout << "(2)  Print  float " << fl1 << " " << fl2 << endl;

12.  cout.precision(4); cout << "(3)  precision 4" << endl;
13.  cout << "(4)   Print  float " << fl1 << " " << fl2 << endl;

14.  cout.precision(7); cout << "(5)  precision 7" << endl;
15.  cout << "(6)   Print  float " << fl1 << " " << fl2 << endl;
16.
17.
18.


19.
20.
21.
22.
23.
24.  return 0;  }
```

```
$ g++ float_precision_1.cc
$ ./a.out
(1)  precision default 6
(2)  Print  float 3.14159 133.142
```


{../L9_TYPES/float_precision_1.cc}

Controlling the Precision: example (1)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include <iomanip>
4.  int main ()
5.  {
6.   float fl1 = 3.1415926 5358979f, fl2 = 133.14159265358979f;
7.   double db1 = 3.14159265358979, db2 = 133.14159265358979;
8.
9.   streamsize ssize = cout.precision();  // store the default precision
10. cout << "(1)  precision default " << ssize << endl;
11. cout << "(2)  Print  float " << fl1 << " " << fl2 << endl;

12. cout.precision(4); cout << "(3)  precision 4" << endl;
13. cout << "(4)   Print  float " << fl1 << " " << fl2 << endl;

14. cout.precision(7); cout << "(5)  precision 7" << endl;
15. cout << "(6)   Print  float " << fl1 << " " << fl2 << endl;
16.
17.
18.


19.
20.
21.
22.
23.
24. return 0;  }
```

```
$ g++ float_precision_1.cc
$ ./a.out
(1)  precision default 6
(2)  Print  float 3.14159 133.142
(3)  precision 4
(4)   Print  float 3.142 133.1
(5)  precision 7
(6)   Print  float 3.141593 133.1416
```


{../L9_TYPES/float_precision_1.cc}

Controlling the Precision: example (1)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include <iomanip>
4.  int main ()
5.  {
6.   float fl1 = 3.14159265358979f, fl2 = 133.14159265358979f;
7.   double db1 = 3.14159265358979, db2 = 133.14159265358979;
8.
9.   streamsize ssize = cout.precision();  // store the default precision
10.  cout << "(1)  precision default " << ssize << endl;
11.  cout << "(2)  Print  float " << fl1 << " " << fl2 << endl;

12.  cout.precision(4); cout << "(3)  precision 4" << endl;
13.  cout << "(4)   Print  float " << fl1 << " " << fl2 << endl;

14.  cout.precision(7); cout << "(5)  precision 7" << endl;
15.  cout << "(6)   Print  float " << fl1 << " " << fl2 << endl;
16.
17.  cout.precision(10); cout << "(7)  precision 10" << endl; //attention: precision up to 10
18.  cout << "(8)   Print  float " << fl1 << " " << fl2 << endl; //precision greater than the one
     of the type of fl1
19.
20.
21.
22.
23.
24.  return 0;  }
```

```
$ g++ float_precision_1.cc
$ ./a.out
(1)  precision default 6
(2)  Print  float 3.14159 133.142
(3)  precision 4
(4)   Print  float 3.142 133.1
(5)  precision 7
(6)   Print  float 3.141593 133.1416
(7)  precision 10
(8)   Print  float 3.141592741 133.1415863
```


{../L9_TYPES/float_precision_1.cc}

Controlling the Precision: example (1)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include <iomanip>
4.  int main ()
5.  {
6.   float fl1 = 3.14159265358979f, fl2 = 133.14159265358979f;
7.   double db1 = 3.14159265358979, db2 = 133.14159265358979;
8.
9.   streamsize ssize = cout.precision();  // store the default precision
10.  cout << "(1)  precision default " << ssize << endl;
11.  cout << "(2)  Print  float " << fl1 << " " << fl2 << endl;

12.  cout.precision(4); cout << "(3)  precision 4" << endl;
13.  cout << "(4)   Print  float " << fl1 << " " << fl2 << endl;

14.  cout.precision(7); cout << "(5)  precision 7" << endl;
15.  cout << "(6)   Print  float " << fl1 << " " << fl2 << endl;
16.
17.  cout.precision(10); cout << "(7)  precision 10" << endl; //attention: precision up to 10
18.  cout << "(8)   Print  float " << fl1 << " " << fl2 << endl; //precision greater than the one
     of the type of fl1
19.
20.
21.
22.
23.
24.  return 0;  }
```

```
$ g++ float_precision_1.cc
$ ./a.out
(1)  precision default 6
(2)  Print  float 3.14159 133.142
(3)  precision 4
(4)   Print  float 3.142 133.1
(5)  precision 7
(6)   Print  float 3.141593 133.1416
(7)  precision 10
(8)   Print  float 3.141592741 133.1415863
```

The used precision is greater than the one of float

Real
- float (32bits, precision: 7 decimal digits)
- double (64bits, precision: 15 decimal digits)
- long double (96bits, precision: at least 15 decimal digits)

Controlling the Precision: example (1)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include <iomanip>
4.  int main ()
5.  {
6.   float fl1 = 3.14159265358979f, fl2 = 133.14159265358979f;
7.   double db1 = 3.141592653 58979, db2 = 133.141592 65358979;
8.
9.   streamsize ssize = cout.precision();  // store the default precision
10.  cout << "(1)  precision default " << ssize << endl;
11.  cout << "(2)  Print  float " << fl1 << " " << fl2 << endl;

12.  cout.precision(4); cout << "(3)  precision 4" << endl;
13.  cout << "(4)   Print  float " << fl1 << " " << fl2 << endl;

14.  cout.precision(7); cout << "(5)  precision 7" << endl;
15.  cout << "(6)   Print  float " << fl1 << " " << fl2 << endl;
16.
17.  cout.precision(10); cout << "(7)  precision 10" << endl; //attention: precision up to 10
18.  cout << "(8)   Print  float " << fl1 << " " << fl2 << endl; //precision greater than the one
     of the type for fl1
19.  cout << "(9)   Print double " << db1 << " " << db2 << endl;
20.
21.  cout.precision(ssize); //restore default precision
22.  cout << "(10) Execution ends by printing float " << fl1 << " " << fl2 << endl;
23.  cout << "(11) Execution ends by printing double " << db1 << " " << db2 << endl;
24.  return 0;  }
```

```
$ g++ float_precision_1.cc
$ ./a.out
(1)  precision default 6
(2)  Print  float 3.14159 133.142
(3)  precision 4
(4)   Print  float 3.142 133.1
(5)  precision 7
(6)   Print  float 3.141593 133.1416
(7)  precision 10
(8)   Print  float 3.141592741 133.1415863
(9)   Print double 3.1415926 54 133.1415927
(10) Execution ends by printing float 3.14159 133.142
(11) Execution ends by printing double 3.14159 133.142
```


{../L9_TYPES/float_precision_1.cc}

Controlling the Precision: example (2a)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include <iomanip>
4.  int main ()
5.  {
6.
7.   float fl1 = 3.14159265358979f;
8.
9.    streamsize ssize = cout.precision();  // store the default precision
10.  cout << "(precision default " << ssize << ") " << fl1 << endl;

11.  cout.precision(4);
12.  cout << "(precision 4)  Print " << fl1 << endl;
13.
14.  float sum=fl1+0.001 ;

15.  cout << "(precision 4)  Print " << fl1 << " " << sum << endl;
16.  cout.precision(5);  cout << "(precision 5)  Print " << fl1 << " " << sum << endl;
17.
18.  return 0;
19. }
```

```
$ g++ float_precision_1.cc
$ ./a.out
(precision default 6) 3.14159
(precision 4)  Print   3.142
```

Controlling the Precision: example (2b)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include <iomanip>
4.  int main ()
5.  {
6.
7.   float fl1 = 3.14159265358979f;
8.
9.    streamsize ssize = cout.precision();  // store the default precision
10.  cout << "(precision default " << ssize << ") " << fl1 << endl;

11.  cout.precision(4);
12.  cout << "(precision 4)  Print " << fl1 << endl;
13.
14.  float sum=fl1+0.001 ;    //3.14159265358979+0.001 = 3.14259265358979

15.  cout << "(precision 4)  Print " << fl1 << " " << sum << endl;
16.  cout.precision(5);  cout << "(precision 5)  Print " << fl1 << " " << sum << endl;
17.
18.  return 0;
19. }
```

```
$ g++ float_precision_1.cc
$ ./a.out
(precision default 6) 3.14159
(precision 4)  Print   3.142
```

Controlling the Precision: example (2c)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include <iomanip>
4.  int main ()
5.  {
6.
7.   float fl1 = 3.14159265358979f;
8.
9.    streamsize ssize = cout.precision();  // store the default precision
10.  cout << "(precision default " << ssize << ") " << fl1 << endl;

11.  cout.precision(4);
12.  cout << "(precision 4)  Print " << fl1 << endl;
13.
14.  float sum=fl1+0.001 ;    //3.14159265358979+0.001 = 3.14259265358979

15.  cout << "(precision 4)  Print " << fl1 << " " << sum << endl;
16.  cout.precision(5);  cout << "(precision 5)  Print " << fl1 << " " << sum << endl;
17.
18.  return 0;
19. }
```

```
$ g++ float_precision_1.cc
$ ./a.out
(precision default 6) 3.14159
(precision 4)  Print   3.142
(precision 4)  Print   3.142          3.143
```

Controlling the Precision: example (2d)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include <iomanip>
4.  int main ()
5.  {
6.
7.   float fl1 = 3.14159265358979f;
8.
9.    streamsize ssize = cout.precision();  // store the default precision
10.  cout << "(precision default " << ssize << ") " << fl1 << endl;

11.  cout.precision(4);
12.  cout << "(precision 4)  Print " << fl1 << endl;
13.
14.  float sum=fl1+0.001 ;    //3.14159265358979+0.001 = 3.14259265358979

15.  cout << "(precision 4)  Print " << fl1 << " " << sum << endl;
16.  cout.precision(5);  cout << "(precision 5)  Print " << fl1 << " " << sum << endl;
17.
18.  return 0;
19. }
```

```
$ g++ float_precision_1.cc
$ ./a.out
(precision default 6) 3.14159
(precision 4)  Print   3.142
(precision 4)  Print   3.142          3.143
(precision 5)  Print   3.1416         3.1426
```

Pay attention to the difference between the representation in the standard output and the one in the memory

Controlling the Precision: example (3)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include <iomanip>

4.  int main ()
5.  {
6.    float fl1 = 3.14159265358979f, fl2 = 133.14159265358979f;
7.    double db1 = 3.14159265358979, db2 = 133.14159265358979;
8.
9.    streamsize ssize = cout.precision();  // store the default precision
10.
11.  cout << "(1)  setprecision to 5" << endl;
12.  cout << "(2)   Print  float " << setprecision(5) << fl1 << " " << fl2 << endl;
13.
14. cout << "(3)  setprecision to 5 + fixed" << endl;  //fixed: normal notation, precision by
    decimal
15. cout << "(4)   Print float " << setprecision(5) << fixed << fl1 << " " << fl2 << endl;
16.
17. cout << "(7)  setprecision to 5 + scientific" << endl; //scientific notation, precision by
    decimal
18. cout << "(8)   Print  float " << setprecision(5) << scientific << fl1 << " " << fl2 << endl;
19.
20.  cout.precision(ssize); //restore default precision
21.  cout << defaultfloat; //restore default notation and precision (both integer and decimal)
22.  cout << "(9) Execution ends by printing float " << fl1 << " " << fl2 << endl;
23.  cout << "(10) Execution ends by printing double " << db1 << " " << db2 << endl;  return 0;
24. }
```

```
$ g++ float_precision_1.cc
$ ./a.out
(1)   setprecision to 5
(2)    Print  float 3.1416 133.14
(3)   setprecision to 5 + fixed
(4)    Print float 3.14159 133.14159
(7)   setprecision to 5 + scientific
(8)    Print  float 3.14159e+00 1.33142e+02
(9)   Execution ends by printing float 3.14159 133.142
(10) Execution ends by printing double 3.14159 133.142
```

Comparing two floating-point numbers: the problem (1)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include <cmath>

4.  int main ()
5.  {
6.    float actual1 = 10 - 9;
7.    cout << "(EX0) actual1= " << actual1<<  endl;

8.    float actual2 = 10.0f - 0.2f * 45;   // 10.0 - 9.0
9.    cout << "(EX0) actual2= " << actual2<<  endl;

10.  float actual3 =  10.0f,  decrement = 0.2f;
11.  int count = 45;
12.  for (int index = 0; index < count; ++index)
13.        actual3 -= decrement;
14.  cout << "(EX0) actual3= " << actual3<<  endl;

15.  if (actual3 == 1.0f)
16.    cout << "(EX0) EQUAL executed" <<  endl;
17.  else
18.    cout << "(EX0) NOT-EQUAL executed" <<  endl; //←printed
19.}
```

```
$ g++ float_comparison.cc
$ ./a.out
(EX0) actual1= 1
(EX0) actual2= 1
(EX0) actual3= 1.00001
(EX0) NOT-EQUAL executed
```

{../L9_TYPES/float_comparison.cc}

Comparing two floating-point numbers: the problem (1)

```
1.  using namespace std;
2.  #include <iostream>
3.  #include <cmath>

4.  int main ()
5.  {
6.    float actual1 = 10 - 9;
7.    cout << "(EX0) actual1= " << actual1<<  endl;

8.    float actual2 = 10.0f - 0.2f * 45;
9.    cout << "(EX0) actual2= " << actual2<<  endl;

10.  float actual3 =  10.0f,  decrement = 0.2f;
11.  int count = 45;
12.  for (int index = 0; index < count; ++index)
13.        actual3 -= decrement;
14.  cout << "(EX0) actual3= " << actual3<<  endl;

15.  if (actual3 == 1.0f)
16.    cout << "(EX0) EQUAL executed" <<  endl;
17.  else
18.    cout << "(EX0) NOT-EQUAL executed" <<  endl; //←printed
19.}
```

```
$ g++ float_comparison.cc
$ ./a.out
(EX0) actual1= 1
(EX0) actual2= 1
(EX0) actual3= 1.00001
(EX0) NOT-EQUAL executed
```

```
$ g++ -Wfloat-equal float_comparison.cc
float_comparison.cc: In function 'int main()':
float_comparison.cc:15:18: warning: comparing floating point with == or != is unsafe [-Wfloat-equal]
   15 |   if (actual3 == 1.0f)
      |                  ^~~~
$ ./a.out
(EX0) actual1= 1
(EX0) actual2= 1
(EX0) actual3= 1.00001
(EX0) NOT-EQUAL executed
```

{../L9_TYPES/float_comparison.cc}

Comparing two floating-point numbers: the problem (2)

```
1. using namespace std;
2. #include <iostream>
3. #include <cmath>

4. int main ()
5. {
6.  …
7.   //EX.1
8.   float epsilon=0.0001f;
9.   bool isEqual= fabs(actual3 - 1.0f) <= epsilon;
10. cout << "(EX1) diff="<< fabs(actual3 - 1.0f) << " eps=" << epsilon << " isEq=" << isEqual << endl;
11. if (isEqual)
12.    cout << "(EX1) EQUAL executed" <<  endl;  //←printed
13. else
14.    cout << "(EX1) NOT-EQUAL executed" <<  endl;
15. }
```

```
$ g++ float_comparison.cc
$ ./a.out
…
(EX1) diff=5.00679e-06 eps=0.0001 isEq=1
(EX1) EQUAL executed
```

{../L9_TYPES/float_comparison.cc}

Comparing two floating-point numbers: the problem (3a)

```
1. using namespace std;
2. #include <iostream>
3. #include <cmath>
4. #include <limits>

5. int main ()
6. {
7.  …
8.   //EX.2
9.   float epsilon=numeric_limits<float>::epsilon(); //←limits by default
10. bool isEqual= fabs(actual3 - 1.0f) <= epsilon;
11. cout << "(EX2) diff="<< fabs(actual3 - 1.0f) << " eps=" << epsilon << " isEq=" << isEqual << endl;
12.  if (isEqual)
13.    cout << "(EX2) EQUAL executed" <<  endl;
14.  else
15.    cout << "(EX2) NOT-EQUAL executed" <<  endl; //←printed

16. }
```

#include <limits>
//epsilon, min, max
e.g., numeric_limits<double>::max()
e.g., numeric_limits<int>::max()

```
$ g++ float_comparison.cc
$ ./a.out
…
(EX2) diff=5.00679e-06 eps=1.19209e-07 isEq=0
(EX2) NOT-EQUAL executed
```

Alt.
#include <cfloat>
//FLT_EPSILON, FLT_MAX, FLT_MIN ..
//DBL_EPSILON, DBL_MAX, DBL_MIN ..

https://en.cppreference.com/w/cpp/types/numeric_limits
https://en.cppreference.com/w/cpp/header/cfloat

{../L9_TYPES/float_comparison.cc}

Comparing two floating-point numbers: the problem (3b)

https://en.cppreference.com/w/cpp/types/numeric_limits
https://en.cppreference.com/w/cpp/header/cfloat

cppreference.com Create account Search cppreference.com

Page Discussion Standard revision: Diff Read View source View hist

C++ Utilities library Type support std::numeric_limits

std::numeric_limits

Defined in header <limits>

```
template< class T > class numeric_limits;
```

The std::numeric_limits class template provides a standardized way to query various properties of arithmetic largest possible value for type int is std::numeric_limits<int>::max()).

This information is provided via specializations of the std::numeric_limits template. The standard library make specializations for all arithmetic types (only lists the specializations for cv-unqualified arithmetic types):

Defined in header <limits>

```
template<> class numeric_limits<bool>;
template<> class numeric_limits<char>;
template<> class numeric_limits<signed char>;
template<> class numeric_limits<unsigned char>;
template<> class numeric_limits<wchar_t>;
```

Details….

List of member constant and functions

radix [static]	the radix of integer base used by the representation of the given type (public static member constant)
min_exponent [static]	one more than the smallest negative power of the radix that is a valid normalized floating-point value (public static member constant)
min_exponent10 [static]	the smallest negative power of ten that is a valid normalized floating-point value (public static member constant)
max_exponent [static]	one more than the largest integer power of the radix that is a valid finite floating-point value (public static member constant)
max_exponent10 [static]	the largest integer power of 10 that is a valid finite floating-point value (public static member constant)
traps [static]	identifies types which can cause arithmetic operations to trap (public static member constant)
tinyness_before [static]	identifies floating-point types that detect tinyness before rounding (public static member constant)

Member functions

min [static]	returns the smallest finite value of the given non-floating-point type, or the smallest positive normal value of the given floating-point type (public static member function)
lowest [static](C++11)	returns the lowest finite value of the given type, i.e. the most negative value for signed types, 0 for unsigned types (public static member function)
max [static]	returns the largest finite value of the given type (public static member function)
epsilon [static]	returns the difference between 1.0 and the next representable value of the given floating-point type (public static member function)
round_error [static]	returns the maximum rounding error of the given floating-point type (public static member function)
infinity [static]	returns the positive infinity value of the given floating-point type (public static member function)
quiet_NaN [static]	returns a quiet NaN value of the given floating-point type (public static member function)
signaling_NaN [static]	returns a signaling NaN value of the given floating-point type (public static member function)
denorm_min [static]	returns the smallest positive subnormal value of the given floating-point type (public static member function)

Helper classes

float_round_style	indicates floating-point rounding modes (enum)
float_denorm_style	indicates floating-point denormalization modes (enum)

Examples

Example

Run this code

```
#include <iostream>
#include <limits>

int main()
{
    std::cout << "type\t| lowest()\t| min()\t\t| max()\n"
              << "bool\t| "
              << std::numeric_limits<bool>::lowest() << "\t\t| "
              << std::numeric_limits<bool>::min() << "\t\t| "
              << std::numeric_limits<bool>::max() << '\n'
              << "uchar\t| "
              << +std::numeric_limits<unsigned char>::lowest() << "\t\t| "
              << +std::numeric_limits<unsigned char>::min() << "\t\t| "
              << +std::numeric_limits<unsigned char>::max() << '\n'
              << "int\t| "
              << std::numeric_limits<int>::lowest() << "\t| "
              << std::numeric_limits<int>::min() << "\t| "
              << std::numeric_limits<int>::max() << '\n'
              << "float\t| "
              << std::numeric_limits<float>::lowest() << "\t| "
              << std::numeric_limits<float>::min() << "\t| "
              << std::numeric_limits<float>::max() << '\n'
              << "double\t| "
              << std::numeric_limits<double>::lowest() << "\t| "
              << std::numeric_limits<double>::min() << "\t| "
              << std::numeric_limits<double>::max() << '\n';
}
```

Mixed operations and type conversion

- Often different types of operands are used in the same expression, or it is assigned to a variable a value of a different type than the one of the variable itself
- In any mixed operation it is always needed a conversion of type that can be implicit or explicit

- E.g.,
  - **int** price = 275000;
  - **double** weight = 0.3;
  - **int** cost = price * weight;

Implicit type conversions

- Implicit type conversions are done by the compiler
- The most significant implicit conversions are:
  - in the evaluation of numeric expressions, operands are converted to the type of that of larger size
  - in the assignment, an expression is always converted to the type of the variable
- E.g.,
  - **float** x1 = 3; //x1 = 3.0 (i.e., not 3), an integer value is assigned to a float variable
  - **float** x2 = 3.0; //a double value is assigned to a float variable
  - **float** x3 = 3.0f; //a float value is assigned to a float variable
  - **int** x4 = 2*3.6; //x4 = 7 (i.e., not 7.2)
  - **int** x5 = 4/3; //x5= 1 (i.e., not 1.33)
  - **float** x6 = 4/3; //x6= 1 (i.e., not 1.33)
  - **float** x7 = 4.0/3.0; //x7=1.333

Examples:

{ ../TYPES/**conversion_1**.cc}

Explicit type conversions

- The programmer can explicit ask to a type conversion: from a type to another one (named casting)
- Two notations exist
  - Prefix
    - E.g., int x1 = (int) 3.14;
  - Functional
    - E.g., double x2 = double(3);

Example

- **float** x6 = 4/3; //x6= 1 (i.e., not 1.33)
- **float** x6 = (double)4/(double)3; //x6= 1.33
- **float** x6 = 4.0/double(3); //x6= 1.33

Conversions of numeric types

- Promotion: conversion from one type to a similar one
  - short ⇒ int ⇒ long ⇒ long long,
  - float ⇒ double ⇒ long double
  - It guarantees to maintain the same value
- Conversion between compatible types
  - Conversion from real to integer: the value is truncated
    - int x = (int) 4.7; ⇒ x==4,
    - int x = (int) -4.7; ⇒ x==-4
  - Conversion from integer to real: the value can lose precision
    - float y = float(2147483600); // 2^{31}-48
    - ⇒ y == 2147483648.0 // 2^{31}

    (note: int -> float, problem – both up to 32bits)

    (note: int -> double, no problem – double has enough number of bits)

Examples: { ../TYPES/conversion_2.cc}

For additional details:
http://www.cplusplus.com/doc/tutorial/typecasting

Example

```
float yif =  float(2147483600);
float yf = 2147483600.00;
double yd = 2147483600.00;
cout << fixed << " 2147483600 ="
cout << yif << "\n" <<yf << "\n" << yd << endl;
```

```
$ ./a.out
2147483600=
2147483648.000000
2147483648.000000
2147483600.000000
```